

Plan d'implémentation du formateur AutoLISP

OpenAI

Principes d'organisation

Le travail doit être découpé en modules indépendants autant que possible, avec une progression incrémentale :

1. obtenir un scanner fiable ;
2. construire un parseur syntaxique léger ;
3. produire un pretty-printer minimal ;
4. ajouter les options de style ;
5. sécuriser par tests ;
6. valider sur corpus réel.

Le projet doit viser une première version utile rapidement, puis durcir la couverture et les cas limites.

Arborescence suggérée

```
src/  
  fmt-main.lsp  
  fmt-options.lsp  
  fmt-io.lsp  
  fmt-scanner.lsp  
  fmt-parser.lsp  
  fmt-cst.lsp  
  fmt-printer.lsp  
  fmt-rules.lsp  
  fmt-comments.lsp  
  fmt-case.lsp  
  fmt-errors.lsp  
  fmt-tests.lsp  
  fmt-golden.lsp  
  fmt-util.lsp  
tests/  
  scanner/  
  parser/
```

```
printer/  
golden/  
corpus/  
docs/  
specifications.org  
plan.org
```

Lots d'implémentation

LOT 0 — cadrage et conventions

Tâches

- figer la spécification des options ;
- figer les noms de modules ;
- définir la représentation des tokens ;
- définir la représentation des nœuds CST ;
- choisir la stratégie de rendu des erreurs ;
- définir les conventions de tests.

Livrables

- conventions internes ;
- schéma des structures de données ;
- jeu d'exemples minimaux.

Critère de fin

Toute l'équipe peut lire et discuter les structures avant codage lourd.

LOT 1 — infrastructure de base

Tâches

- module utilitaire (fonctions listes, chaînes, positions) ;
- module I/O (lecture/écriture fichiers) ;
- structure de résultat / erreur ;
- infrastructure de tests simple ;
- fonction de comparaison de sorties exactes.

Tests

- lecture d'un fichier texte simple ;
- écriture d'un fichier ;
- remontée d'erreur standardisée.

Critère de fin

Le squelette du projet s'exécute et permet d'ajouter les modules suivants proprement.

LOT 2 — scanner / tokenizer

Objectif

Transformer le texte source en tokens fiables, avec gestion correcte des commentaires et chaînes.

Tâches

- implémenter un curseur de lecture caractère par caractère ;
- reconnaître espace, tab, newline ;
- reconnaître (et) ;
- reconnaître quote ;
- reconnaître chaînes ;
- reconnaître commentaires de ligne ;
- reconnaître commentaires bloc ;| ... | ;
- reconnaître atomes ;
- stocker ligne/colonne ou offset ;
- produire une API `scan-string` et `scan-file`.

Jeux de tests

- tokens sur expressions simples ;
- parenthèses dans chaînes ;
- parenthèses dans commentaires ;
- bloc ;| ... | ; multi-ligne ;
- symboles `:foo`, `pk!`, `vl-catch-all-apply` ;
- dotted pair.

Critère de fin

Le scanner passe tous les tests unitaires et produit des tokens utilisables par le parseur.

LOT 3 — parseur léger / CST

Objectif

Reconstruire la structure des s-expressions et attacher les trivia sans évaluation.

Tâches

- définir nœuds atome, chaîne, liste, quote, trivia ;

- parser les formes top-level ;
- parser les listes imbriquées ;
- gérer les quotes ;
- attacher trivia préfixe/suffixe ;
- détecter parenthèses non équilibrées ;
- détecter fin de chaîne ou commentaire bloc non terminé.

Jeux de tests

- forme simple ;
- plusieurs formes top-level ;
- commentaires avant/après forme ;
- quote simple et quote de liste ;
- erreur de parenthèse ;
- erreur de commentaire bloc non fermé.

Critère de fin

Le parseur produit un CST suffisamment précis pour permettre le pretty-printing.

LOT 4 — pretty-printer minimal

Objectif

Produire un rendu stable sans encore traiter toutes les subtilités de style.

Tâches

- moteur d'indentation générique ;
- émission des atomes ;
- émission des chaînes ;
- émission des listes ;
- rendu top-level multi-formes ;
- stratégie simple pour commentaires préservés ;
- normalisation minimale des lignes vides.

Jeux de tests

- liste plate ;
- liste imbriquée ;
- fichier multi-formes ;
- commentaires préservés ;
- test d'idempotence simple.

Critère de fin

Un corpus simple peut déjà être reformatté de façon stable.

LOT 5 — styles de parenthèses

Objectif

Ajouter les modes `cl` et `nail`.

Tâches

- isoler la politique d'émission des fermetures ;
- implémenter style `cl` ;
- implémenter style `nail` ;
- ajouter l'option utilisateur correspondante ;
- golden tests comparatifs.

Jeux de tests

- mêmes entrées, sorties attendues différentes selon style ;
- `if/progn` imbriqués ;
- `defun` avec code interne multi-niveau ;
- style `nail` sur code déjà `nail` ;
- conversion `nail` -> `cl` -> `nail` stable.

Critère de fin

Le passage d'un style à l'autre est fiable et déterministe.

LOT 6 — règles d'indentation spécifiques

Objectif

Améliorer la lisibilité des formes courantes AutoLISP.

Tâches

- définir table de règles par tête de forme ;
- implémenter règles pour `defun` ;
- `lambda` ;
- `if` ;
- `cond` ;
- `setq` ;
- `progn` ;
- `while` ;
- `repeat` ;
- `foreach` ;
- prévoir extensibilité future.

Jeux de tests

- un golden test par forme ;

- formes imbriquées ;
- listes d'arguments avec / pour variables locales.

Critère de fin

Le rendu est lisible sur des fonctions réelles d'équipe.

LOT 7 — normalisation de casse des symboles

Objectif

Ajouter les modes `preserve`, `downcase`, `upcase`.

Tâches

- identifier précisément les tokens transformables ;
- ne jamais modifier chaînes ni commentaires ;
- implémenter transformation des symboles ;
- intégrer au pipeline d'émission.

Jeux de tests

- symboles simples ;
- symboles avec : ;
- symboles avec !, *, - ;
- vérification de non-modification des chaînes ;
- vérification de non-modification des commentaires.

Critère de fin

Les trois modes donnent les sorties attendues sans altérer le texte non symbolique.

LOT 8 — politique de commentaires

Objectif

Supporter `preserve` et normalisation inspirée CL.

Tâches

- classer les commentaires selon contexte ;
- mode `preserve` ;
- mode `cl` ;
- conversion ;| ... | ; -> lignes préfixées ;
- option contextuelle ;
- alignement des commentaires de fin de ligne.

Jeux de tests

- commentaire top-level ;
- commentaire de section ;
- commentaire dans sexp ;
- commentaire de fin de ligne ;
- conversion d'un bloc multi-ligne ;
- alignement sur colonne.

Critère de fin

Le rendu des commentaires est stable et conforme aux options choisies.

LOT 9 — interface utilisateur et mode check

Objectif

Rendre l'outil utilisable par l'équipe et la CI.

Tâches

- parser les options ;
- support `--check` ;
- support `--in-place` ;
- support sortie vers fichier ;
- codes retour cohérents ;
- messages d'erreur lisibles.

Jeux de tests

- check fichier conforme ;
- check fichier non conforme ;
- formatage in place ;
- sortie vers fichier ;
- combinaison d'options.

Critère de fin

L'outil est utilisable dans des scripts et pipelines d'équipe.

LOT 10 — corpus réel et non-régression

Objectif

Valider sur code AutoLISP existant et figer des golden tests réalistes.

Tâches

- constituer un corpus représentatif ;

- sélectionner fichiers simples, moyens, difficiles ;
- exécuter le formateur ;
- inspecter les écarts ;
- corriger règles ;
- figer sorties de référence.

Jeux de tests

- golden corpus complet ;
- double passage = idempotence ;
- validation manuelle d'un échantillon.

Critère de fin

L'équipe peut lancer l'outil sur son code sans corruption et avec confiance raisonnable.

Tests détaillés à prévoir

Scanner

- expression minimale ;
- espaces et tabulations ;
- fins de ligne CR/LF si nécessaire ;
- commentaire ligne ;
- commentaire bloc ;
- chaîne contenant ; ;
- chaîne contenant parenthèses ;
- symboles exotiques ;
- dotted pairs.

Parseur

- listes simples ;
- formes top-level multiples ;
- quote ;
- commentaire entre deux formes ;
- erreur EOF dans liste ;
- erreur EOF dans chaîne ;
- erreur EOF dans commentaire bloc.

Printer

- indentation de base ;
- rendu top-level ;
- rendu des commentaires ;

- style `cl` ;
- style `nail` ;
- stabilité des lignes vides.

Transformations

- preserve/downcase/upcase ;
- commentaires preserve/cl ;
- conversion bloc -> semicolons ;
- inline comments column.

Intégration

- scan + parse + print sur petit corpus ;
- check mode ;
- in place ;
- idempotence ;
- non-régression.

Dépendances entre lots

- LOT 0 précède tout ;
- LOT 1 précède LOT 2 à 10 ;
- LOT 2 précède LOT 3 ;
- LOT 3 précède LOT 4 ;
- LOT 4 précède LOT 5 à 8 ;
- LOT 5, 6, 7, 8 peuvent partiellement avancer en parallèle une fois LOT 4 stabilisé ;
- LOT 9 dépend des options déjà stabilisées ;
- LOT 10 est continu mais se fige en fin de projet.

Jalons suggérés

Jalon A — noyau syntaxique

Inclut LOT 0 à 3.

Résultat : scanner + parseur utilisables.

Jalon B — formatage minimal exploitable

Inclut LOT 4 et LOT 5.

Résultat : le projet sait déjà reformater les parenthèses dans les deux styles.

Jalon C — lisibilité équipe

Inclut LOT 6, 7, 8.

Résultat : formatage réellement utile au quotidien.

Jalon D — industrialisation

Inclut LOT 9 et LOT 10.

Résultat : intégration outillage / CI / corpus réel.

Critères d'achèvement globaux

Le projet est terminé lorsque :

- tous les lots critiques sont implémentés ;
- les golden tests sont stables ;
- l'idempotence est démontrée ;
- un corpus réel d'équipe passe correctement ;
- les options documentées sont supportées ;
- l'outil est suffisamment robuste pour être adopté en pré-commit ou en revue de code.